Architecture Vocabulary Card

> **Day 1 · Activity 1 handout.** Today's shared reference. You need this vocabulary to follow the architect's reasoning — not to win arguments by quoting taxonomy.

Term	What it means	TPM-relevant signal
Monolith	One deployable unit; one repo; one runtime	Faster start; harder to scale teams
Modular monolith	Monolith with strict internal module boundaries	Best of both for many cases; underrated
Microservice	Independently deployable, owned by one team	Independent scaling and failure domains; high ops cost
Service-oriented (SOA)	Coarse-grained services; often shared infra	Often what "microservices" actually means in practice
Distributed monolith	Multiple deployables that *must* deploy together	Worst of both — anti-pattern
Function-as-a-service	Per-function deploys (e.g., Azure Functions)	Bursty workloads; not a default
Event-driven	Components communicate via events / queues	Decouples timing; complicates debugging

---

The three-question frame

When someone proposes "let's build it as a microservice," ask three questions:

1. **Whose deployment cadence does this protect?** Microservices are right when *another team's* deployment cadence would block this team's. If only one team owns the work, a separate service mostly buys ops cost.
2. **What independent failure domain do we want?** A separate service is justified when failures should be isolated. If the feature *can't function* without its dependencies anyway, separation buys no resilience.
3. **What scaling axis are we anticipating?** If 90% of load lives on 10% of features with a different scale curve, separation lets you scale them independently. If load is uniform, one runtime is cheaper.

> If at least **two of three** answers are "yes, with evidence" — a microservice is plausible. If two are "we don't know" — start with a modular monolith.